

Components, Props, Lists & Keys

1. Component

A Component is a JS function that returns a JSX element.

```
const Welcome = () => <h1 className="message">Hello, User</h1>;
```

The component name should always start with a capital letter as react treats the components starting with lowercase letters as HTML elements.

```
<script type="text/babel">
```

```
  const Welcome = () => <h1 className="message">Hello, User</h1>;
```

```
  ReactDOM.render(<Welcome />, document.getElementById("root"));
```

```
</script>
```

We can call the function with self-closing tags as shown above

```
<Welcome />.
```

1.1 Properties (Props)

React allows us to pass information to a component using props.

1.1.1 Passing Props

We can pass props to any component as we declare attributes for any HTML element.

Syntax: <Component propName1="propValue1" propName2="propValue2" />

```
const Welcome = () => <h1 className="message">Hello, User</h1>;
```

```
ReactDOM.render(  
  
  <Welcome name="Rahul" greeting="Hello" />,  
  
  document.getElementById("root")  
  
);
```

1.1.2 Accessing Props

The components accept props as parameters and can be accessed directly.

Syntax:const Component = (props) => {

```
  // We can access props here  
  
};  
  
const Welcome = (props) => {  
  
  const { name, greeting } = props;  
  
  return (  
  
    <h1 className="message">  
  
      {greeting}, {name}  
  
    </h1>  
  
  );  
  
};
```

```
ReactDOM.render(  
  
  <Welcome name="Rahul" greeting="Hello" />,  
  
  document.getElementById("root")  
  
);
```

1.2 Component is Reusable

A Component is a piece of reusable code that can be used in various parts of an application.

Example:

```
const Welcome = (props) => {  
  
  const { name, greeting } = props;  
  
  return (  
  
    <h1 className="message">  
  
      {greeting}, {name}  
  
    </h1>  
  
  );  
  
};
```

```
ReactDOM.render(  
  

```

```
<div>

  <Welcome name="Rahul" greeting="Hello" />

  <Welcome name="Ram" greeting="Hi" />

</div>,

document.getElementById("root")

);
```

1.3 Component is Composable

We can include a component inside another component.

```
const Welcome = (props) => {

  const { name, greeting } = props;

  return (

    <h1 className="message">

      {greeting}, {name}

    </h1>

  );

};

const Greetings = () => (

  <div>
```

```
<Welcome name="Rahul" greeting="Hello" />
```

```
<Welcome name="Ram" greeting="Hi" />
```

```
</div>
```

```
);
```

```
ReactDOM.render(<Greetings />, document.getElementById("root"));
```

2. Third-party Packages

Creating a real-world app involves lot of setup because a large number of components need to be organised.

Facebook has created a third-party package,

create-react-app, to generate a ready-made React application setup.

2.1 React + Tailwind CSS Setup with Vite

⚙️ Why Vite?

- ⚡ Ultra-fast dev server
- ✨ Native ES Module support
- 📦 Smaller production bundles

-  Works great with Tailwind

Step-by-Step: Set Up React + Tailwind CSS using Vite

♦ 1. Create a Vite + React App

```
npm create vite@latest my-app --template react
```

```
cd my-app  
npm install
```

Install Tailwind CSS

```
bash  
CopyEdit  
npm install -D tailwindcss postcss autoprefixer  
npx tailwindcss init -p
```

Configure Tailwind

In `tailwind.config.js`, set up the paths:

```
js  
CopyEdit  
/** @type {import('tailwindcss').Config} */  
export default {  
  content: [  
    "./index.html",  
    "./src/**/*.{js,ts,jsx,tsx}",  
  ],  
  theme: {
```

```
    extend: {},  
  },  
  plugins: [],  
}
```

◆ 4. Add Tailwind to your CSS

In `src/index.css`, replace the contents with:

```
css  
CopyEdit  
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```

Ensure this file is imported in `main.jsx`:

```
js  
CopyEdit  
import React from 'react'  
import ReactDOM from 'react-dom/client'  
import App from './App.jsx'  
import './index.css' // Make sure this is here!  
  
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>,  
)
```

◆ 5. Run Your App

```
bash  
CopyEdit  
npm run dev
```

This will start the Vite dev server, usually at <http://localhost:5173>.

2.1.2 React Application Folder Structure

- `public/` folder: Where we will keep assets like images, icons, videos etc
- `src/` folder: Where we will do the majority of our work. All of our React components will be placed here.
- `node_modules`
- `package-lock.json`

node_modules:

This directory contains dependencies and sub-dependencies of packages used by the current react app, as specified by `package.json`.

package-lock.json:

This file contains the exact dependency tree installed in

`node_modules`. This provides a way to ensure every team member has the same version of dependencies and sub-dependencies.

The

`index.js` in the path `src/` folder/ is a starting point to the application. `App.js`, `App.css` are imported in this file.

2.2 Pre-Configured tools

The

`create-react-app` comes pre-configured with:

- **Live editing:** Allows React components to be live reloaded.
- **ESLint:** Analyzes source code to report programming errors, bugs, and syntax errors.

- **Prettier**: Enforces a consistent style for indentation, spacing, semicolons and quotes, etc.
- **Babel**: Compiles JSX into Regular JavaScript
- **Webpack**: Stitches together a group of modules into a single file (or group of files). This process is called Bundling.

3. Keys

Keys help React to identify which items have changed, added, or removed. They should be given to the elements inside the array for a stable identity.

The best way to pick a

key is to use a string that uniquely identifies a list item among its siblings. Most often, we would use IDs (uniqueNo) from our data as keys.

Example:const userDetails = [

```
{  
  
  uniqueNo: 1,  
  
  imageUrl:  
  
    'https://assets.ccbp.in/frontend/react-js/esther-howard-img.png',  
  
  name: 'Esther Howard',  
  
  role: 'Software Developer'  
  
}
```

```
]
```

File: src/[App.js](#)

```
import UserProfile from './components/UserProfile/index'
```

```
const userDetailsList = [
```

```
{
```

```
  uniqueNo: 1,
```

```
  imageUrl:
```

```
    'https://assets.ccbp.in/frontend/react-js/esther-howard-img.png',
```

```
  name: 'Esther Howard',
```

```
  role: 'Software Developer'
```

```
},
```

```
{
```

```
  uniqueNo: 2,
```

```
  imageUrl:
```

```
    'https://assets.ccbp.in/frontend/react-js/floyd-miles-img.png',
```

```
  name: 'Floyd Miles',
```

```
  role: 'Software Developer'
```

```
},
```

```
{
```

```
    uniqueNo: 3,  
  
    imageUrl:  
  
    'https://assets.ccbp.in/frontend/react-js/jacob-jones-img.png',  
  
    name: 'Jacob Jones',  
  
    role: 'Software Developer'  
  
  },  
  
  {  
  
    uniqueNo: 4,  
  
    imageUrl:  
  
    'https://assets.ccbp.in/frontend/react-js/esther-devon-lane.png',  
  
    name: 'Devon Lane',  
  
    role: 'Software Developer'  
  
  }  
  
]
```

```
const App = () => (  
  
  <div className="list-container">  
  
    <h1 className="title">Users List</h1>
```

```
<ul>

  {userDetailsList.map((eachItem) => (

    <UserProfile userDetails={eachItem} />

  )))}

</ul>

</div>

)

export default App
```

The

index.js file in the folder is considered to be the main file in it. So, a path like ./components/UserProfile can be used instead of ./components/UserProfile/index.

Keys used within arrays should be unique among their siblings. However, they don't need to be unique in the entire application.

3.1 Keys as Props

Keys don't get passed as a prop to the components.

```
const UserProfile = props => {

  const {userDetails} = props

  const {imageUrl, name, role, key} = userDetails
```

```
console.log(key) // undefined
```

```
return (
```

```
<li className="user-card-container">
```

```
<img src={imageUrl} className="avatar" alt="avatar" />
```

```
<div className="user-details-container">
```

```
<h1 className="user-name"> {name} </h1>
```

```
<p className="user-designation"> {role} </p>
```

```
</div>
```

```
</li>
```

```
)
```

```
}
```

```
export default UserProfile
```

If we need the same value in the component, pass it explicitly as a prop with a different name.

Example:

```
const UsersList = userDetailsList.map((userDetails) =>
```

```
<UserProfile
```

```
key={userDetails.uniqueNo}
```

```
        uniqueNo={userDetails.uniqueNo}

        name={userDetails.name} />

    );
```

4. Users List Application

File: src/App.js

```
import UserProfile from './components/UserProfile/index'

const userDetailsList = [

  {

    uniqueNo: 1,

    imageUrl:

      'https://assets.ccbp.in/frontend/react-js/esther-howard-img.png',

    name: 'Esther Howard',

    role: 'Software Developer'

  },

  {

    uniqueNo: 2,

    imageUrl:

      'https://assets.ccbp.in/frontend/react-js/floyd-miles-img.png',
```

```
    name: 'Floyd Miles',

    role: 'Software Developer'

  },

  {

    uniqueNo: 3,

    imageUrl:

      'https://assets.ccbp.in/frontend/react-js/jacob-jones-img.png',

    name: 'Jacob Jones',

    role: 'Software Developer'

  },

  {

    uniqueNo: 4,

    imageUrl:

      'https://assets.ccbp.in/frontend/react-js/esther-devon-lane.png',

    name: 'Devon Lane',

    role: 'Software Developer'

  }

]
```

```
const App = () => (  
  
  <div className="list-container">  
  
    <h1 className="title">Users List</h1>  
  
    <ul>  
  
      {userDetailsList.map((eachItem) => (  
  
        <UserProfile userDetails={eachItem} />  
  
      ))}  
  
    </ul>  
  
  </div>  
  
)  
  
export default App  
  
import './index.css'
```

```
const UserProfile = props => {  
  
  const {userDetails} = props  
  
  const {imageUrl, name, role} = userDetails
```

```
return (  
  
  <li className="user-card-container">  
  
    <img src={imageUrl} className="avatar" alt="avatar" />  
  
    <div className="user-details-container">  
  
      <h1 className="user-name"> {name} </h1>  
  
      <p className="user-designation"> {role} </p>  
  
    </div>  
  
  </li>  
  
)  
}
```

export default UserProfile

React Error - ENOSPC: System limit for the number of file watchers reached

Since **create-react-app** live-reloads and recompiles files on save, it needs to keep track of all project files.

To fix the error, run the below commands in the terminal:

Insert the new value into the system config

1. `echo fs.inotify.max_user_watches=524288 | sudo tee -a /etc/sysctl.conf && sudo sysctl -p`

Check that the new value was applied

2. `cat /proc/sys/fs/inotify/max_user_watches`

Config variable name (not runnable)

3. `fs.inotify.max_user_watches=524288`